SireSoft · Leader in Innovation

SireSoft Scan Tool

Project Documentation — AI receipt scanning on the company server

Version 1.0 | English | http://10.0.2.2:9000

Prepared by
SireSoft Engineering

Date
October 2026

Audience
Team, HR, new developers

Contents

1 Introduction
2 Goals and objectives
3 Scope
4 Quick start: links and login
5 Users, roles and use cases
6 Requirements
7 System architecture
8 How the project works
9 Where the data is stored
10 Django Administration
11 Looking inside PostgreSQL
12 24-hour automatic deletion
13 Project structure
14 API endpoints
15 Security and privacy
16 Testing
17 Deployment and operations
18 Troubleshooting
19 Future goals
20 Glossary

1 Introduction

SireSoft Scan Tool is a web application that reads receipts and bills automatically. A user uploads a photo or PDF of a receipt, and the system extracts the shop name, date, every line item, taxes and the total, then stores the result in a PostgreSQL database.

The reading is done by an AI vision model that runs **on the company's own server**. No receipt is sent to the internet or to any outside company. The whole system is written in **Python** with the **Django** web framework.

> **In one sentence:** open `http://10.0.2.2:9000`, log in, upload a receipt, and within a minute its data appears in your History — and it is deleted automatically 24 hours later.

Who this document is for

Anyone who needs to use, explain, run or extend the project — including people who have never seen it before. Every term is explained, and the glossary in section 20 covers the technical words.

2 Goals and objectives

Goal: replace manual typing of receipt data with a fast, private, self-hosted scanning service for SireSoft staff.

#	Objective	How it is met
O1	Read receipts automatically	Local AI vision model (Ollama, qwen2.5vl:3b) turns each image into structured data
O2	Keep company data private	Model, database and files all stay on server 10.0.2.2
O3	Each user sees only their own data	Every page and API filters receipts by owner; only the admin sees all
O4	Do not keep data longer than needed	Every receipt is deleted 24 hours after upload, with a live countdown
O5	Let mistakes be fixed	Users can edit extracted values, re-scan, or delete a receipt
O6	Give the admin full visibility	History shows all users; Django Administration shows the raw database

3 Scope

In scope

- Email sign-up, login, logout, password reset
- Upload of JPEG, PNG, WEBP, GIF and PDF receipts (up to 5 files, 10 MB each)
- AI extraction of vendor, address, date, time, items, subtotal, tax, discount, other charges, grand total, payment method and short insights
- History, detail view, edit, corrected copy, re-scan, delete
- Admin view of all receipts; Django Administration
- Automatic deletion of each receipt 24 hours after upload
- Deployment on the company Linux server at 10.0.2.2:9000

Out of scope (today)

- Access from outside the company VPN
- Mobile app
- Export to Excel or accounting software
- Multi-page PDF reading (only page 1 is read)
- Handwritten receipts with very faint print
- Long-term archiving of receipts

4 Quick start: links and login

Connect to the company VPN, then type this address in any browser:

`http://10.0.2.2:9000`

What you want	Link to enter
Log in	`http://10.0.2.2:9000/auth/login/`
Create an account	`http://10.0.2.2:9000/auth/register/`
Upload receipts	`http://10.0.2.2:9000/`
Receipt History	`http://10.0.2.2:9000/history/`
Django Administration	`http://10.0.2.2:9000/admin/`
All receipts stored in PostgreSQL	`http://10.0.2.2:9000/admin/receipts/receipt/`
All users and their emails	`http://10.0.2.2:9000/admin/auth/user/`

Admin login

Where	Username / email	Password
App login page	`admin@siresoft.com`	The admin password, stored as `ADMIN_PASSWORD` in the server's `.env` file. Ask the project owner.
Django Administration	`admin` or `admin@siresoft.com`	

> **VPN required.** 10.0.2.2 is a private address inside the company network. Without the VPN the page will not open.

5 Users, roles and use cases

Role	Can do	Sees
User	Sign up, upload, view, edit, re-scan and delete their own receipts	Only their own receipts
Admin	Everything a user can, plus Django Administration	Every user's receipts, with an "Uploaded By" column
System	Reads receipts with the AI model; deletes receipts after 24 hours	—

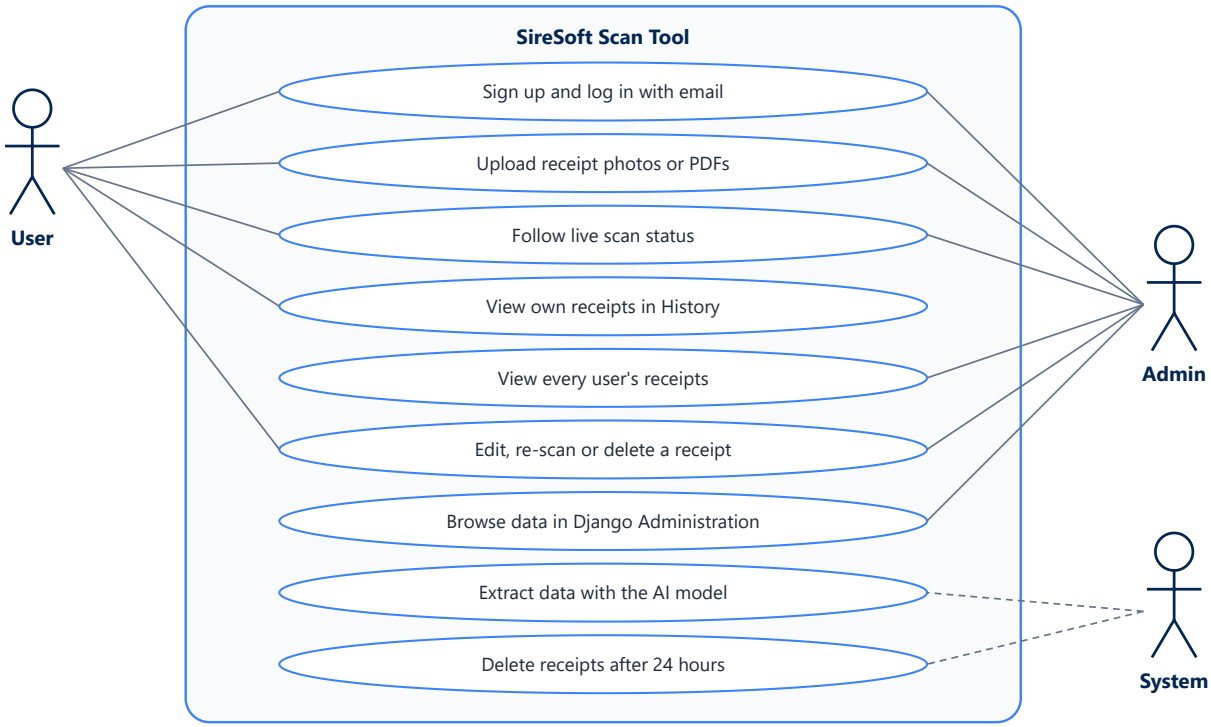


Figure 1 — Use case diagram

6 Requirements

Functional requirements

ID	Requirement
FR-1	A visitor can create an account with an email and password, and log in with that email.
FR-2	A user can upload up to 5 receipt files at once (JPEG, PNG, WEBP, GIF, PDF; 10 MB each).
FR-3	The system extracts vendor, date, line items, totals, payment method and insights from each receipt.
FR-4	The upload page shows each receipt's status live: queued, reading, done or failed.
FR-5	A user sees only their own receipts; the admin sees all receipts.
FR-6	A user can edit extracted values; the system then draws a corrected copy of the receipt.
FR-7	A user can re-scan or delete a receipt.
FR-8	Every receipt, of every user including the admin, is deleted 24 hours after its upload, from History, the database and the disk.
FR-9	History shows a live countdown ring with the time left for each receipt.
FR-10	The admin can browse users and receipts in Django Administration.
FR-11	A user can reset a forgotten password by email.

Non-functional requirements

ID	Quality	Requirement
NFR-1	Privacy	No receipt or extracted data leaves the company server.
NFR-2	Security	Passwords are stored hashed; every page needs login; users cannot open others' receipts.
NFR-3	Availability	Services start on boot and restart automatically after a crash (systemd).
NFR-4	Responsiveness	Uploads return immediately; reading happens in the background worker.
NFR-5	Data retention	Receipts are removed within about one minute of their 24-hour deadline.
NFR-6	Maintainability	Automated test suite (63 tests); all settings in one `.env` file.

Software and hardware requirements

Component	Used in this project
Operating system (server)	Linux server **lsnet**, address 10.0.2.2
Programming language	Python 3
Web framework	Django (4.2 – 5.1) with Django REST Framework
Database	PostgreSQL (driver psycopg2)
Web server	Gunicorn, managed by systemd; static files by WhiteNoise
AI model	Ollama running `qwen2.5vl:3b` (3.2 GB, works on CPU)
Other libraries	PyMuPDF (PDF to image), Pillow (images), requests, python-dotenv
Client	Any modern browser (Chrome, Edge, Firefox) on the company VPN
Hardware	Enough RAM for the model (about 4 GB free) and disk space for uploads

7 System architecture

The project has five parts, all on one server. The **website** answers the browser, the **worker** does the slow AI reading in the background, **PostgreSQL** keeps the data, the **AI model** reads images, and the **media folder** keeps the uploaded files.

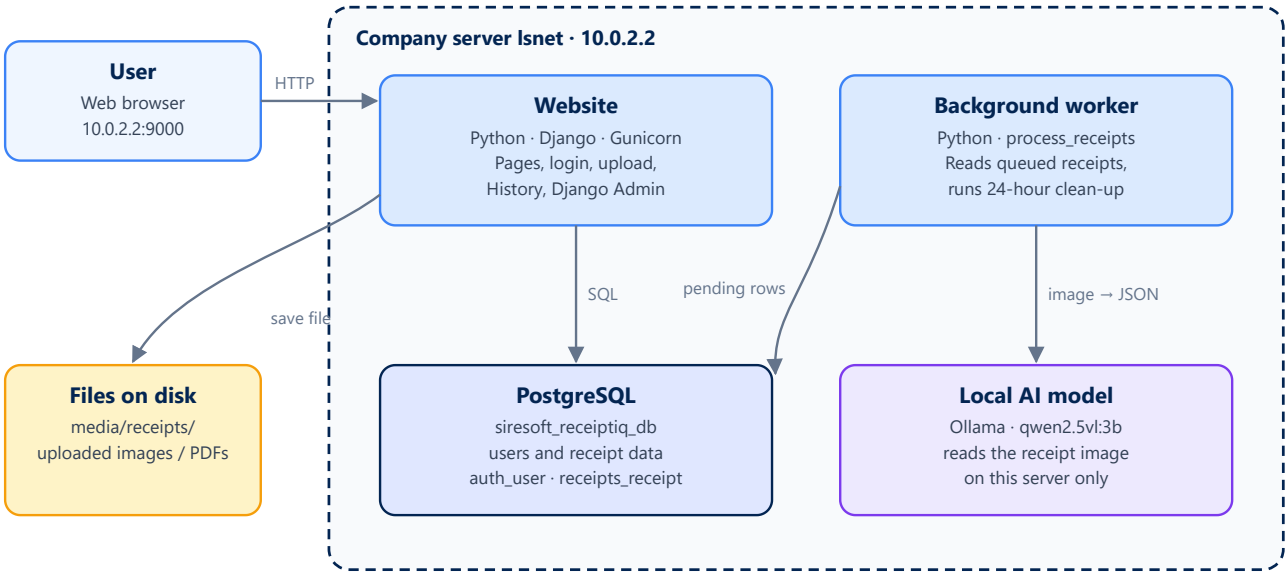


Figure 2 — Architecture of SireSoft Scan Tool

Front end and back end

Layer	Technology	What it contains
Front end	HTML (Django templates), CSS, JavaScript	Login and sign-up pages, upload page with drag-and-drop and live status, History table with countdown rings, receipt detail and edit pages
Back end	Python, Django, Django REST Framework	Accounts, receipt storage, permissions, upload API, AI extraction service, background worker, 24-hour clean-up
Database	PostgreSQL	Users, receipts, extracted data, login sessions
AI	Ollama + qwen2.5vl:3b	Reads the receipt image and returns JSON

8 How the project works

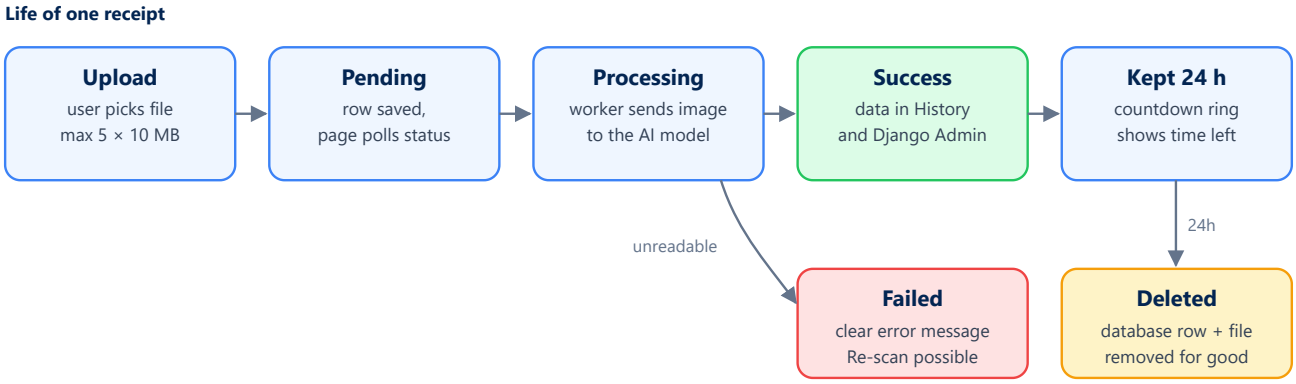


Figure 3 — Lifecycle of a receipt

1. **Upload.** The user drops files on the upload page. The website checks type and size, saves each file to `media/receipts/`, and creates a row in the database with status *pending*. The page returns at once.
2. **Queue.** The upload page asks the server every few seconds how each receipt is doing (`/api/receipts/status/`).
3. **Processing.** The background worker takes the oldest pending receipt, marks it *processing*, turns a PDF into an image if needed, shrinks large images, and sends the image to the AI model with clear instructions.
4. **Result.** The model answers with JSON. The worker cleans it (numbers, missing fields, totals) and saves it. Status becomes *success*, or *failed* with a readable message.
5. **Use.** The receipt appears in History and in Django Administration. The user can view, edit (a corrected picture is drawn), re-scan or delete it.
6. **Clean-up.** Exactly 24 hours after upload, the receipt row and its files are deleted. The countdown ring in History shows how much time is left.

> **Why a background worker?** A local AI model can take from seconds to several minutes per receipt. Doing that inside the web request would freeze the page; the worker lets the website stay fast.

9 Where the data is stored

Text data goes to **PostgreSQL**; the receipt files themselves go to a **folder on the server's disk**.

What	Where exactly
Database server	PostgreSQL on 10.0.2.2
Database name	`siresoft_receiptiq_db`
User accounts and emails	Table `auth_user`
Receipts and extracted data	Table `receipts_receipt`
Login sessions	Table `django_session`
Uploaded files	`/home/siresoft/siresoft-scan-tool/media/receipts/`
Corrected pictures	`/home/siresoft/siresoft-scan-tool/media/receipts/corrected/`
Connection settings	`DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT` in `/home/siresoft/siresoft-scan-tool/.env`

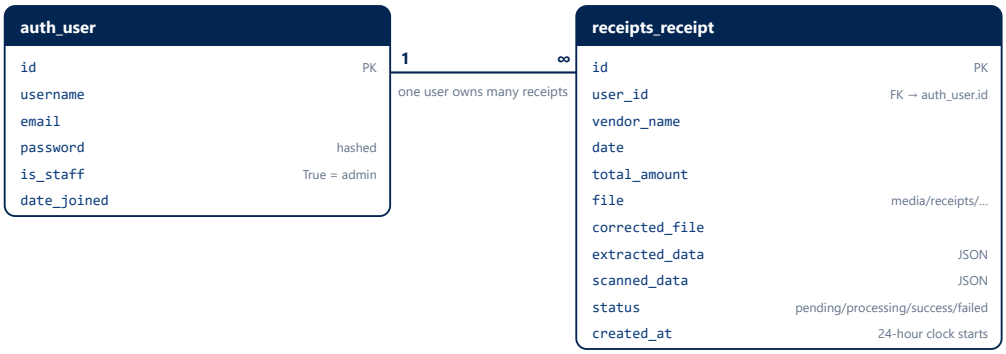


Figure 4 — Database design (entity relationship)

What is inside `extracted_data`

`scanned_data` keeps the AI's first reading unchanged, so after an edit the system can show each value before and after.

```
{
  "establishment": { "name": "Y GENERATION SYSTEMS", "address": "...", "date": "25-Sep-2024",
"time": "..." },
  "items": [ { "name": "...", "quantity": 1, "unit_price": 3213.0, "total": 3213.0 } ],
  "bill_summary": { "subtotal": 0.0, "tax": 0.0, "discount": 0.0, "other_lines": [],
                    "grand_total": 3213.0, "payment_method": "cash" },
  "insights": [ "one short observation about this receipt" ]
}
```

10 Django Administration

Django Administration is a ready-made admin website that reads the same PostgreSQL tables. It is the easiest way to confirm that data was stored.

1. Connect to the VPN and open `http://10.0.2.2:9000/admin/`.
2. Log in with username `admin` (or `admin@siresoft.com`) and the admin password.
3. Under **Authentication and Authorization**, click **Users** — every account, its email and sign-up date (table `auth_user`).
4. Under **Receipts**, click **Receipts** — every receipt with vendor, total, status, owner and upload time (table `receipts_receipt`).
5. Click any receipt to see its full extracted data and the exact file path on the server.
6. Use the filters on the right (status, date, user) and the search box (vendor) to narrow the list.

> **Read-only by design.** Receipts cannot be edited in Django Administration, because the worker or the user would overwrite the change. Edit receipts from the app's History page instead.

11 Looking inside PostgreSQL

For a direct look at the tables, log in to the server and open the database:

```
# on your laptop
ssh siresoft@10.0.2.2
# on the server
sudo -u postgres psql siresoft_receiptiq_db
```

```
-- all users and their emails
SELECT id, email, date_joined, is_staff FROM auth_user ORDER BY date_joined DESC;

-- latest 20 receipts with owner
SELECT r.id, u.email, r.vendor_name, r.total_amount, r.status, r.created_at, r.file
FROM receipts_receipt r JOIN auth_user u ON u.id = r.user_id
ORDER BY r.created_at DESC LIMIT 20;

-- receipts per user
SELECT u.email, COUNT(r.id) FROM auth_user u
LEFT JOIN receipts_receipt r ON r.user_id = u.id GROUP BY u.email;

\q   -- leave psql
```

To see the stored files: `ls -lh ~/siresoft-scan-tool/media/receipts/`

12 24-hour automatic deletion

Every receipt has its own 24-hour clock that starts at the moment it is uploaded. When the clock reaches zero, the receipt is removed from History, from PostgreSQL (so also from Django Administration) and its file is deleted from the server.

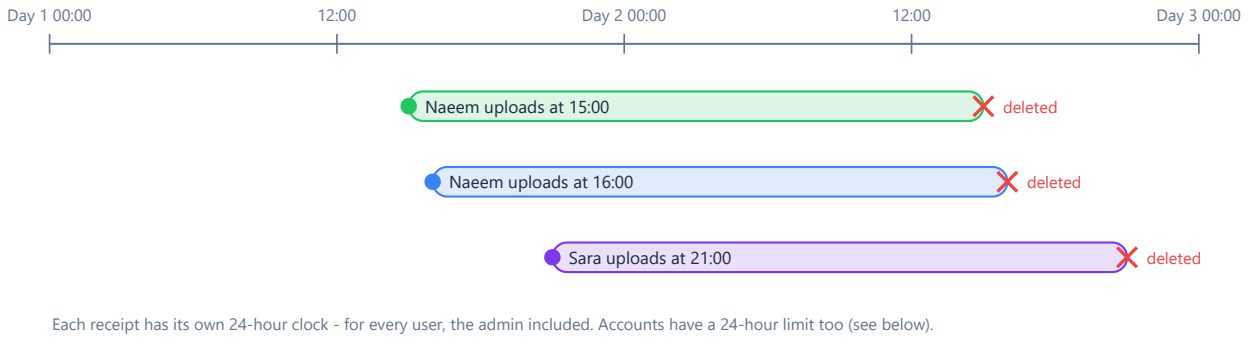


Figure 5 — Each receipt is deleted 24 hours after its own upload

Rule	Detail
Clock starts	At upload time (`created_at`)
Deleted after	24 hours (setting `USER_DATA_TTL_HOURS=24` in `.env`)
What is deleted	Database row, uploaded file, corrected picture
Who it applies to	Every receipt of every user, the admin included
What is kept	Only the admin account (admin@siresoft.com)
How often it runs	Every minute, inside the worker and the website — no cron job needed
Run by hand	`./venv/bin/python manage.py purge_expired_receipts`

The countdown ring

History shows a coloured ring for each receipt next to a clock such as **23:54:12** that counts down every second. The ring is **green** normally, **amber** under 3 hours and **red** under 1 hour.

24-hour limit on sign up and sign in

Every user who signs up can sign in for **24 hours after signup** only. The clock is separate for each user and starts at signup.

- Signed up at 6:00 → can sign in until 6:00 the next day. Signed up at 9:00 → until 9:00 the next day.
- After 24 hours the correct password no longer works, and a user who is still signed in is signed out on their next click.
- The account (username, email, password) and its receipts are deleted from PostgreSQL. The login page says: “Your 24-hour access has ended and the account was deleted. Please sign up again to continue.”
- The same email can sign up again and gets a new 24 hours.
- **The admin has no limit.** The admin credentials stay saved in the server’s .env and the admin account is never deleted.

13 Project structure

Folder / file	Purpose
`apps/accounts/`	Sign-up, login, password reset, admin account command (`ensure_admin`), 24-hour clean-up (`retention.py`), email login for Django Admin
`apps/receipts/`	Receipt model, pages (upload, History, detail, edit), upload API, AI service (`services.py`), worker (`process_receipts`), corrected copy drawing
`config/`	Django settings and main URL list
`templates/`	HTML pages
`static/`	CSS, JavaScript, logos
`media/`	Uploaded receipt files (created at run time, not in Git)
`deploy/`	Server start scripts and systemd service files
`docs/`	This documentation and the server command sheet
`.env`	Secrets and settings: database, admin account, AI model, port (never committed to Git)
`requirements.txt`	Python packages to install

14 API endpoints

The upload page talks to these endpoints. All require login and only return the caller's own receipts (all receipts for the admin).

Method	Address	Purpose
POST	`/api/upload/`	Upload 1–5 receipt files
GET	`/api/receipts/status/?ids=1,2`	Live status of the given receipts
GET	`/api/receipts/`	List of receipts
GET	`/api/receipts/<id>/`	One receipt with extracted data

15 Security and privacy

- Every page except login, sign-up and password reset requires a logged-in user.
- Passwords are stored as one-way hashes by Django, never as plain text.
- Receipts are always looked up through one rule (`visible_to`): staff see all, everyone else only their own. Opening another user's receipt address returns "not found".
- Forms are protected against cross-site request forgery (CSRF). A stale form (back button, resubmission) is sent to a fresh page instead of an error.
- Secrets (database password, admin password, secret key) live only in the server's `.env` file, which is excluded from Git.
- The AI model runs locally; no receipt is sent to an external service.
- Receipts and signed-up accounts are kept only 24 hours, reducing the personal data stored.

16 Testing

The project has an automated test suite of **63 tests** that run against a temporary test database. Run it from the project folder:

```
python manage.py test apps
```

Test group	What it checks
LoginViewTests, RegisterTests	Login by email or username, sign-up rules, duplicate emails
EnsureAdminTests	The admin account is created and repaired from `.env`
RetentionTests	Each receipt is deleted 24 hours after its own upload; files removed; the admin's receipts too; accounts end 24 h after signup; admin can log in to Django Admin with email
ReceiptVisibilityTests	Users cannot see or open other users' receipts; admin sees all
ReceiptFileTests, ReceiptPreviewTests	Files are served safely and PDFs get a picture preview
ReceiptEditTests, CorrectedCopyAndDeleteTests	Editing values, drawing the corrected copy, deleting receipts and their files
ExtractionDetailsTests, RescanTests	Cleaning the AI's answer and re-scanning a receipt

Three login-page tests currently error on Python 3.14 because of a known incompatibility in Django's test client; this does not affect the running application.

17 Deployment and operations

Item	Value
Server	lsnet, `10.0.2.2`, user `siresoft`
Address	`http://10.0.2.2:9000`
Project folder	`/home/siresoft/siresoft-scan-tool`
Services	`siresoft-scan-tool` (website), `siresoft-scan-tool-worker` (AI worker)
Code repository	GitHub: Engr-Mahnoor-code/SireSoft-Scan-Tool (private)

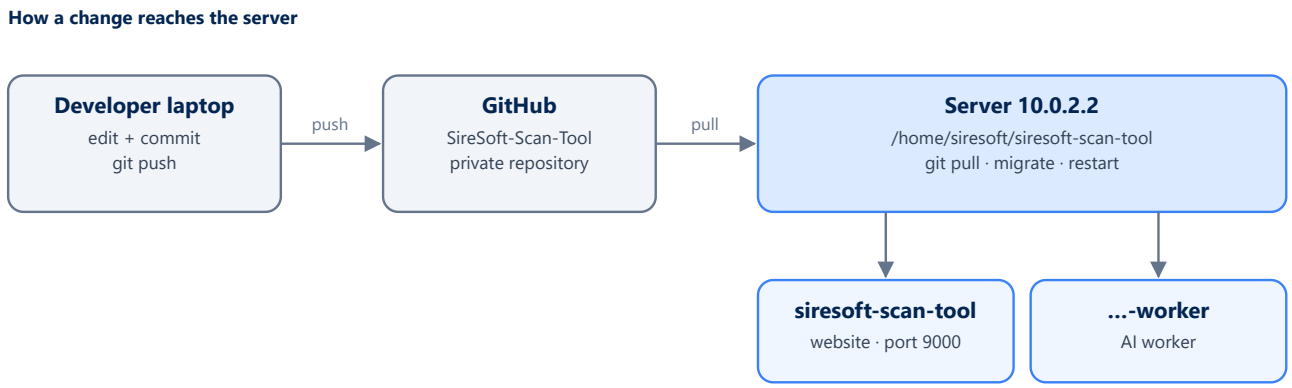


Figure 6 — From code change to running server

Daily commands (on the server)

```
# is it running? expect "active" twice
systemctl is-active siresoft-scan-tool siresoft-scan-tool-worker

# restart
sudo systemctl restart siresoft-scan-tool siresoft-scan-tool-worker

# logs
journalctl -u siresoft-scan-tool -n 50 --no-pager
journalctl -u siresoft-scan-tool-worker -n 50 --no-pager
```

Publishing a code change

```
# 1. on the laptop (prompt: DC@Mahnoor MINGW64)
git add -A
git commit -m "describe the change"
git push origin main

# 2. on the server (prompt: [siresoft@lsnet ...]$)
ssh siresoft@10.0.2.2
cd ~/siresoft-scan-tool
git pull
./venv/bin/python manage.py migrate
./venv/bin/python manage.py collectstatic --noinput
sudo systemctl restart siresoft-scan-tool siresoft-scan-tool-worker
```

18 Troubleshooting

Problem	Likely cause	Fix
Page does not open	VPN is off	Connect to the VPN, then open http://10.0.2.2:9000
10.0.2.2:9000 shows a different app	Another service holds port 9000	`sudo ss -tlnp | grep :9000`, stop that service, restart the Scan Tool
Service shows "activating"	Website cannot start	Read the website log
"Server Error (500)"	Error inside the app	Last lines of the website log name the cause
Receipt stays "Queued"	Worker or Ollama not running	Check the worker log and `ollama list`
Receipt "Failed"	Image unreadable or blurred	Upload a clearer photo, or Re-scan
New styling not visible	Static files not collected	Run `collectstatic`, restart, press Ctrl+Shift+R
"No such file or directory"	Command typed on the laptop, not the server	Run `ssh siresoft@10.0.2.2` first
Cannot log in as admin	Password in database differs from .env	`./venv/bin/python manage.py ensure_admin`

19 Future goals

Idea	Benefit
Secure public address (HTTPS domain such as receipts.siresoft.com)	Use the tool without the VPN
Export to Excel / CSV	Hand data to accounts in one click
Read every page of multi-page PDFs	Complete invoices, not only page 1
Dashboard of totals per vendor and month	Spending insight for management
Configurable retention per user or team	Keep some receipts longer when needed
Larger model on a GPU	Faster and more accurate reading of faint receipts
Mobile-friendly camera upload	Scan a receipt straight from a phone

20 Glossary

Term	Meaning
Python	The programming language the whole project is written in
Django	A Python framework for building websites; provides login, database access and the admin site
Django Administration	Django's built-in admin website at /admin/ for looking at stored data
PostgreSQL	The database program that stores users and receipts in tables
Table / row / column	A table is like a spreadsheet; each receipt is one row; each field is a column
Ollama	A program that runs AI models on our own server
qwen2.5vl:3b	The vision AI model that reads receipt images
Gunicorn	The program that runs the Django website on port 9000
systemd service	Linux feature that starts programs on boot and restarts them if they stop
Worker	A background program that processes receipts one by one
IP address / port	10.0.2.2 is the server's address; 9000 is the "door" the website listens on
VPN	Secure connection into the company network, needed to reach 10.0.2.2
.env file	Private settings file on the server (passwords, database name)
Git / GitHub	Version control for the code / the website that stores the repository
JSON	A text format for structured data, used for the extracted receipt data